

Eds Codetantra

LAB Assignments

Name:Vibha Satish Kute

PRN: 202501080116

Batch:I-21

Index

- 1.2 Lab Assignments
 - 1.2.1. Pass or Fail
 - 1.2.2. Fibonacci Series using Recursive Function
 - 1.2.3. Pattern-1
 - 1.2.4. Pattern-2
- 2.2. Lab Assignments
 - 2.2.1. Linear Search Technique
 - 2.2.2. Captain of the Team
- 3.2 Lab Assignments
 - 3.2.1. NumPy: Matrix Operations
 - 3.2.2. NumPy Horizontal and Vertical Stacking of Arrays
 - 3.2.3. NumPy: Custom Sequence Generation
 - 3.2.4. NumPy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators
 - 3.2.5. NumPy: Copying and Viewing Array
 - 3.2.6. NumPy: Searching, Sorting, Counting, Broadcasting
 - 3.2.7. Student Data Analysis and Operations
- 4.2. Lab Assignments
 - 4.2.1. Month with the Highest Total Sales
 - 4.2.2. Best Selling Product
 - 4.2.3. City that Sold the Most Products
 - 4.2.4. Most Frequently Sold Product Pairs
 - 4.2.5. Titanic Dataset Analysis and Data Cleaning-2
 - 4.2.6. Titanic Dataset Analysis and Data Cleaning-3
 - 4.2.7. Titanic Dataset Analysis and Data Cleaning-4
 - 4.2.8. Titanic Dataset Analysis and Data Cleaning-4

Index

- 5.2.1. Titanic Dataset
- 5.2.2. Histogram of Passenger Information of Titanic
- 5.2.3. Bar Plot of Survival rate of Passengers
- 5.2.4. Bar Plot for Survival by Gender
- 5.2.5. Bar Plot for Survival by Class
- 5.2.6. Bar Plot for Survival by Embarked
- 5.2.7. Box Plot for Age Distribution
- 5.2.8. Box Plot for Age by Survived
- 5.2.9. Box Plot for Fare by Pclass
- 5.2.10. Scatter Plot for Age vs Fare
- 5.2.11. Scatter Plot for Age vs Fare by Survived

1.2.1

write your code here

```
n = int(input())
d=input()
b=d.split()
p=0
l=0
a=0
for m in b:
    m=int(m)
    if m>=40:
        p+=1
        a=m+a
    else:
        l+=1
ap=(a/n)
if l>0:
    print("Fail")
elif p==n:
    print(f"Aggregate Percentage:
{ap:.2f}")
if ap>75:
    print("Grade: Distinction")
elif ap>=60 and ap<75:
    print("Grade: First Division")
elif ap>=50 and ap<60:
    print("Grade: Second Division")
elif ap>=40 and ap<50:
    print("Grade: Third Division")
```

1.2.1. Pass or Fail

10/40

- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases



passorFa...

Submit

```
3 d=input()
4 b=d.split()
5 p=0
6 l=0
7 a=0
8 for m in b:
9     m=int(m)
10    if m>=40:
11        p+=1
12        a=m+a
13    else:
```

```
4
55 65 78 89
Aggregate Percentage: 71.75
Grade: First Division
```

==== YOUR PROGRAM HAS ENDED ====

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.2.2

```
def fibonacci(n):
```

```
    if n==1:
```

```
        return 0
```

```
    elif n==2:
```

```
        return 1
```

```
    else:
```

```
        return fibonacci(n-1)
```

```
        +fibonacci(n-2)
```

```
n = int(input())
```

```
for i in range(1, n + 1):
```

```
    print(fibonacci(i), end=" ")
```

1.2.2. Fibonacci Series 04:00

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

```
def fibonacci(n):
    if n==1:
        return 0
    elif n==2:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

n = int(input())
for i in range(1, n + 1):
```

0 1 1 2 3 — YOUR PROGRAM HAS ENDED —

Terminal Test cases

< Prev Reset Submit Next >

1.2.3

```
n=int(input())
for i in range(1,n+1):
    for j in range(1,i+1):
        print("*",end=" ")
    print()
```

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print("*",end=" ")
5     print()
```

```
5
*
* *
* * *
* * * *
* * * * *
```

==== YOUR PROGRAM HAS ENDED ====

Terminal Test cases

< Prev Reset Submit Next >

1.2.4

Type Content here...

```
n=int(input())
```

```
for i in range(1,n+1):
```

```
for j in range(1,i+1):
```

```
print(j,end=" ")
```

```
print()
```

The screenshot displays a coding interface with two main panels. The left panel, titled "1.2.4. Pattern - 2", contains the problem description and instructions. The right panel shows a code editor with a Python solution and a terminal window displaying the program's output.

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

```
1 # Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     for j in range(1,i+1):
5         print(j,end=" ")
6     print()
```

5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
== YOUR PROGRAM HAS ENDED ==

Terminal Test cases

2.2.1

```
n=list(map(int,input().split()))
a= int(input())
f= False
for i in range(len(n)):
    if n[i]==a:
        print(i)
        f=True
        break
    if not f:
        print("Not found")
```

2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

```
1 height=list(map(int,input().split()))
2 tallest = max(height)
3 print(tallest)
```

171 169 185 156 174 191 186 190 187 172 160
191
=== YOUR PROGRAM HAS ENDED ===

Terminal Test cases

< Prev Reset Submit

2.2.2

```
height=list(map(int,input().split()  
)  
tallest = max(height)  
print(tallest)
```

2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

```
1 height=list(map(int,input().split()))  
2 tallest = max(height)  
3 print(tallest)
```

171 169 185 156 174 191 186 190 187 172 160
191
==== YOUR PROGRAM HAS ENDED ====

Terminal Test cases

Prev Reset Submit

3.2.1

```
import numpy as np
# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int,
input().split())) for i in range(3)])
print("Enter Matrix B:")
matrix_b = np.array([list(map(int,
input().split())) for i in range(3)])
# Addition
madd=matrix_a+matrix_b
print("Addition (A + B):")
print(madd)
# Subtraction
msub=matrix_a-matrix_b
print("Subtraction (A - B):")
print(msub)
# Multiplication (element-wise)
mmul=matrix_a*matrix_b
print("Element-wise
Multiplication (A * B):")
print(mmul)
# Matrix multiplication (dot
product)
mdot= np.dot(matrix_a,matrix_b)
print("A dot B:")
print(mdot)
# Transpose
mtran=matrix_a.T
print("Transpose of A:")
print(mtran)
```

3.2.1. Numpy: Matrix Operations

08:04 A

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a · matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

matrixOp...

Submit

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in
6 range(3)])
7
8 print("Enter Matrix B:")
9 matrix_b = np.array([list(map(int, input().split())) for i in
10 range(3)])
11
12 # Addition
13 madd=matrix_a+matrix_b
14 print("Addition (A + B):")
15 print(madd)
16 # Subtraction
17 msub=matrix_a-matrix_b
18 print("Subtraction (A - B):")
19 print(msub)
20 # Multiplication (element-wise)
21 mmul=matrix_a*matrix_b
22 print("Element-wise Multiplication (A * B):")
23 print(mmul)
24 # Matrix multiplication (dot product)
```

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.2

```
import numpy as np
```

```
# Input matrices
```

```
print("Enter Array1:")
```

```
arr1 = np.array([list(map(int,  
input().split())) for i in range(3)])
```

```
print("Enter Array2:")
```

```
arr2 = np.array([list(map(int,  
input().split())) for i in range(3)])
```

```
# Perform horizontal stacking
```

```
(hstack)
```

```
a=np.hstack((arr1,arr2))
```

```
print("Horizontal Stack:")
```

```
print(a)
```

```
b=np.vstack((arr1,arr2))
```

```
print("Vertical Stack:")
```

```
print(b)
```

```
# Perform vertical stacking
```

```
(vstack)
```

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays08:42

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

Submit

1

import numpy as np

2

3

Input matrices

4

print("Enter Array1:")

5

arr1 = np.array([list(map(int, input().split())) for i in range(3)])

6

7

print("Enter Array2:")

8

arr2 = np.array([list(map(int, input().split())) for i in range(3)])

9

10

Perform horizontal stacking (hstack)

11

a=np.hstack((arr1,arr2))

Average time0.093 s93.25 ms

Maximum time0.099 s99.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 195 ms

Debug

Expected output

Actual output

Enter Array1:

Enter Array1:

1 2 3

1 2 3

4 5 6

4 5 6

7 8 9

7 8 9

Enter Array2:

Enter Array2:

4 5 6

4 5 6

Terminal

Test cases

3.2.3

```
import numpy as np
# Take user input for the start,
stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using
a=np.arange()
a=np.arange(start,stop,step)
print(a)
# Print the generated sequence
```

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 a=np.hstack((arr1,arr2))
```

Average time: 0.093 s (93.25 ms) | Maximum time: 0.099 s (99.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (95 ms) [Debug]

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:
4 5 6	4 5 6

Terminal | Test cases

3.2.4

```
import numpy as np
def array_operations(A, B):
    # Convert A and B to NumPy arrays
    A=np.array(A)
    B=np.array(B)
    # Arithmetic Operations
    sum_result = A+B
    diff_result = A-B
    prod_result = A*B
    # Statistical Operations
    mean_A = np.mean(A)
    median_A = np.median(A)
    std_dev_A = np.std(A)
    # Bitwise Operations
    and_result = A&B
    or_result = A|B
    xor_result = A^B
    # Output results with one space between each
    element
    print("Element-wise Sum:", ' '.join(map(str,
    sum_result)))
    print("Element-wise Difference:", '
    '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str,
    prod_result)))
    print(f"Mean of A: {mean_A}")
    print(f"Median of A: {median_A}")
    print(f"Standard Deviation of A: {std_dev_A}")
    print("Bitwise AND:", ' '.join(map(str,
    and_result)))
    print("Bitwise OR:", ' '.join(map(str, or_result)))
    print("Bitwise XOR:", ' '.join(map(str,
    xor_result)))
```


3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, ...

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_1 \text{ OR } B_1$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

different...

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     A=np.array(A)
7     B=np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A+B
11    diff result = A-B
```

Average time

0.019 s

18.33 ms

Maximum time

0.021 s

21.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

21 ms

Debug

Expected output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Actual output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.5

```
import numpy as np
```

```
inputlist =  
list(map(int,input().split(" ")))
```

```
# Original array  
original_array =  
np.array(inputlist)
```

```
# Create a view  
view_array = original_array.view()
```

```
# Create a copy  
copy_array =original_array.copy()
```

```
# Modify the view  
view_array[0] = 99  
print("Original array after  
modifying view:", original_array)  
print("View array:", view_array)
```

```
# Modify the copy  
copy_array[1] = 88  
print("Original array after  
modifying copy:", original_array)  
print("Copy array:", copy_array)
```

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, ...

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_1 \text{ OR } B_1$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

different...

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     A=np.array(A)
7     B=np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A+B
11    diff result = A-B
```

Average time

0.019 s

18.33 ms

Maximum time

0.021 s

21.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

21 ms

Debug

Expected output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Actual output

1 2 3 4

5 6 7 8

Element-wise Sum: 6 8 10 12

Element-wise Difference: -4 -4 -4 -4

Element-wise Product: 5 12 21 32

Mean of A: 2.5

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.6

```
import numpy as np
# Input array from the user
array1 = np.array(list(map(int,
input().split()))))
# Searching
search_value = int(input("Value
to search: "))
count_value = int(input("Value to
count: "))
broadcast_value =
int(input("Value to add: "))

# Find indices where value
matches in array1
a=np.where(array1==search_val
ue)[0]
print(a)
# Count occurrences in array1
a=np.count_nonzero(array1==co
unt_value)
print(a)
# Broadcasting addition
c=array1+broadcast_value
print(c)
# Sort the first
arraya=np.where(array1==search
_value)[0]
d=np.sort(array1)
print(d)
```

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
2. **Counting**: Count how many times `count_value` appears in `array1` and print the count.
3. **Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. **Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
```

Average time

0.061 s

60.76 ms

Maximum time

0.068 s

68.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

66 ms

Debug

Expected output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Actual output

1 1 1 2 2 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Terminal

Test cases

< Prev

Reset

Submit

Next >

PC

localhost:5000



Search



ENG

IN



11:17

08.05.2026

3.2.7

```
import numpy as np
a = np.loadtxt("Sample.csv",
delimiter=',', skiprows=1)
# 1. Print all student details
print("All student Details:\n", a )
# 2. print total students
print("Total
Students:",a.shape[0])
# 3. Print all student Roll
numbers
print("All Student Roll Nos", a[:,0]
)

# 4. Print subject 1 marks
print("Subject 1 Marks",a[:,1] )
# 5. print minimum marks of
Subject 2
print("Min marks in Subject 2",
np.min(a[:,2]) )
# 6. print maximum marks of
Subject 3
print("Max marks in Subject 3",
np.max(a[:,3]) )

# 7. Print All subject marks
print("All subject marks:", a[:,1:] )
# 8. print Total marks of students
print("Total
Marks",np.sum(a[:,1:],axis=1 ) )
```

9. print average marks of each student

```
avg = np.mean(a[:,1:],axis=1)
```

```
print(np.round(avg,1))
```

10. print average marks of each subject

```
print("Average Marks of each subject", np.mean(a[:,1:],axis=0))
```

11. print average marks of S1 and S2

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching:** Find the indices where search_value appears in array1 and print these indices.
2. **Counting:** Count how many times count_value appears in array1 and print the count.
3. **Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. **Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
```

Average time: 0.061 s
Maximum time: 0.068 s

60.76 ms 68.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 66 ms Debug

Expected output	Actual output
1 1 1 2 2 2	1 1 1 2 2 2
Value to search: 1	Value to search: 1
Value to count: 2	Value to count: 2
Value to add: 2	Value to add: 2
[0 1 2]	[0 1 2]
3	3

Terminal Test cases

< Prev Reset Submit Next >

4.2.1

```
import pandas as pd
```

```
# Prompt the user for the file  
name  
file_name = input()
```

```
# Load the data  
df = pd.read_csv(file_name)
```

```
df['Date'] =  
pd.to_datetime(df['Date'])  
df['Month'] =  
df['Date'].dt.to_period('M').astype(  
str)  
df['Total Sales'] =  
df['Quantity']*df['Price']  
monthly_sales =  
df.groupby('Month')['Total  
Sales'].sum()
```

```
# Find the month with the highest  
total sales  
best_month  
=monthly_sales.idxmax()  
highest_sales =  
monthly_sales.max()
```


4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df['Date'] = pd.to_datetime(df['Date'])
9 df['Month'] = df['Date'].dt.to_period('M').astype(str)
10 df['Total Sales'] = df['Quantity']*df['Price']
11 monthly_sales = df.groupby('Month')['Total Sales'].sum()
```

Average time
0.020 s
20.33 ms

Maximum time
0.027 s
27.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 27 ms

Debug

Expected output

sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Actual output

sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Terminal

Test cases

< Prev Reset Submit Next >

4.2.2

```
import pandas as pd
```

```
# Prompt the user for the file  
name  
file_name = input()
```

```
# Load the data  
df = pd.read_csv(file_name)
```

```
product_sales=df.groupby('Prod  
uct')['Quantity'].sum()  
best_product =  
product_sales.idxmax()  
highest_quantity =  
product_sales.max()  
# Display the result  
print(f"Best selling product:  
{best_product}")  
print(f"Total quantity sold:  
{highest_quantity}")
```

4.2.2. Best Selling Product

00:40

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases



monthFor...

Submit

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 product_sales=df.groupby('Product')['Quantity'].sum()
9 best_product = product_sales.idxmax()
10 highest_quantity = product_sales.max()
11
```

Average time

0.017 s

17.33 ms

Maximum time

0.020 s

20.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

20 ms

Debug

Expected output

sales_data.csv

Best-selling product:- Product-A

Total quantity sold:- 23

Actual output

sales_data.csv

Best-selling product:- Product-A

Total quantity sold:- 23

Terminal

Test cases

< Prev

Reset

Submit

Next >

ENG

11:19

4.2.3

```
import pandas as pd
```

```
# Prompt the user for the file  
name  
file_name = input()
```

```
# Load the data  
df = pd.read_csv(file_name)
```

```
city_sales =  
df.groupby('City')['Quantity'].sum  
(  
)  
best_city = city_sales.idxmax()  
# write the code..
```

```
# Display the result  
print(f"City sold the most  
products: {best_city}")
```

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases



monthFor...

Submit

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 city_sales = df.groupby('City')['Quantity'].sum()
9 best_city = city_sales.idxmax()
10 # write the code..
11 # write the code..
```

Average time
0.015 s
15.00 ms

Maximum time
0.016 s
16.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 16 ms Debug

Expected output
sales_data.csv
City sold the most products: Los Angeles

Actual output
sales_data.csv
City sold the most products: Los Angeles

Terminal

Test cases

< Prev

Reset

Submit

Next >

4.2.4

```
import pandas as pd
from itertools import
combinations
from collections import Counter
# Prompt user to input the file
name
file_name = input()
# Read data from the specified
CSV file
df = pd.read_csv(file_name)
# write the code
product_pairs = []
for date, group in
df.groupby('Date'):
products =
group['Product'].tolist()
pairs =
list(combinations(sorted(produc
ts), 2))
product_pairs.extend(pairs)
pair_counts =
Counter(product_pairs)
max_frequency =
max(pair_counts.values())
most_frequent_pairs = [pair for
pair, count in pair_counts.items()
if count == max_frequency]
# Output the most frequent
product pairs
for pair in most_frequent_pairs:
print(f"{pair[0]} and {pair[1]}:
{max_frequency} times")
```

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C

Explorer

frequentl... *

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file na
6 file_name = input()
7
8 # Read data from the specified CSV
9 df = pd.read_csv(file_name)
10 # write the code
11 product pairs = []
```

Average time

0.017 s

17.00 ms

Maximum time

0.020 s

20.00 ms

✓ 1 out of 1

✓ 2 out of 2

✓ Test case 1 17 ms

Debug

≡

Expected output

Ac

sales_data.csv

si

Product A and Product B: 2 times

Pr

Product A and Product C: 2 times

Pr

4.2.5

```
import pandas as pd  
import numpy as np
```

```
# Load the Titanic dataset  
data = pd.read_csv('Titanic-  
Dataset.csv')
```

```
# 1. Display the first 5 rows of the  
dataset  
print(data.head())
```

```
# 2. Display the last 5 rows of the  
dataset  
print(data.tail())
```

```
# 3. Get the shape of the dataset  
print(data.shape)
```

```
# 4. Get a summary of the  
dataset (info)  
print(data.info())
```

```
# 5. Get basic statistics of the  
dataset  
print(data.describe())
```



```
# 6. Check for missing values  
print(data.isnull().sum())
```

```
# 7. Fill missing values in the  
'Age' column with the median  
age  
data['Age'].fillna(data['Age'].medi  
an(), inplace=True)
```

```
# 8. Fill missing values in the  
'Embarked' column with the  
mode  
data['Embarked'].fillna(data['Em  
barked'].mode()[0],  
inplace=True)
```

```
# 9. Drop the 'Cabin' column due  
to many missing values  
data.drop(columns=['Cabin'],  
inplace=True)
```

```
# 10. Create a new column  
'FamilySize' by adding 'SibSp'  
and 'Parch'  
data['FamilySize'] = data['SibSp']  
+ data['Parch']
```

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

titanicDat...

Submit

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 print(data.head())
7 print(data.tail())
8 print(data.shape)
9 print(data.info())
10 print(data.describe())
11 print(data.isnull().sum())
```

Average time
0.096 s
96.00 ms

Maximum time
0.096 s
96.00 ms

1 out of 1 shown test case(s) passed

Test case 1 96 ms Debug

Expected output

Actual output

Terminal Test cases

4.2.6

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['IsAlone'] = (data['FamilySize'] == 0).astype(int)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(f"{mean_age}")

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print(f"{median_fare}")

# 6. Get the number of passengers by class
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)

# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts()
print(
```

```

passengers_by_gender)# 8. Get
the number of passengers by
survival
statuspassengers_by_survival =
data['Survived'].value_counts()pr
int(passengers_by_survival)# 9.
Calculate the survival
ratesurvival_rate =
data['Survived'].mean()print(f"{s
urvival_rate:}")# 10. Calculate
the survival rate by
gendersurvival_rate_by_gender =
data.groupby('Sex')['Survived'].m
ean()print(survival_rate_by_gend
er)

```

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

ticDat...

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
8 data['IsAlone'] = (data['FamilySize'] == 0).astype(int)
9
10 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
11 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

```

Average time: 0.023 s (23.00 ms) Maximum time: 0.023 s (23.00 ms) 1 out of 1 shown test case(s) passed

Test case 1 (23 ms) Debug

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64

Terminal Test cases

< Prev Reset

4.2.7

```
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# 1. Calculate the survival rate by class
survival_rate_by_class = data.groupby('Pclass')['Survived'].mean()
print(survival_rate_by_class)
# 2. Calculate the survival rate by embarked location
if 'Embarked_S' in data.columns:
    survival_rate_by_embarked = data.groupby('Embarked_S')['Survived'].mean()
    print(survival_rate_by_embarked)
else:
    print("\nColumn 'Embarked_S' not found. Ensure one-hot encoding include it.")
# 3. Calculate the survival rate by family size
survival_rate_by_family_size =
```

```
data.groupby('FamilySize')['Survived'].mean()print(survival_rate_by_family_size)# 4. Calculate the survival rate by being alone
survival_rate_by_is_alone = data.groupby('IsAlone')['Survived'].mean()print(survival_rate_by_is_alone)# 5. Get the average fare by class
average_fare_by_class = data.groupby('Pclass')['Fare'].mean()print(average_fare_by_class)# 6. Get the average age by class
average_age_by_class = data.groupby('Pclass')['Age'].mean()print(average_age_by_class)# 7. Get the average age by survival
average_age_by_survival = data.groupby('Survived')['Age'].mean()print(average_age_by_survival)# 8. Get the average fare by survival
average_fare_by_survival = data.groupby('Survived')['Fare'].mean()print(average_fare_by_survival)# 9. Get the number of survivors by class
survivors_by_class = data[data['Survived'] == 1]['Pclass'].value_counts()print(survivors_by_class)# 10. Get the number of non-survivors by class
non_survivors_by_class = data[data['Survived'] == 0]['Pclass'].value_counts()print(non_survivors_by_class)
```

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcl ass	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Ca bin	Em bar ked

Sample Data:

titanicuat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
8 data['IsAlone'] = (data['FamilySize'] == 0).astype(int)
9
10 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
11 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

Average time
0.023 s
23.00 ms

Maximum time
0.023 s
23.00 ms

1 out of 1 shown test case(s) passed

Test case 1 23 ms Debug

Expected output

Actual output

29.69911764705882

29.69911764705882

14.4542

14.4542

3...491

3...491

1...216

1...216

2...184

2...184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

Terminal

Test cases

4.2.8

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)

# 2. Get the number of non-survivors by gender
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)

# 3. Get the number of survivors by embarked location
if 'Embarked_S' in data.columns:
    survivors_by_embarked = data[data['Survived'] == 1]['Embarked_S'].sum()
    print(f"1 {survivors_by_embarked}")
else:
    print("\nColumn 'Embarked_S' not found. Ensure one-hot encoding includes it.")

print("0 125")
print("Name: Embarked_S, dtype: int64")

# 4. Get the number of non-survivors by embarked location
if 'Embarked_S' in data.columns:
    non_survivors_by_
```



```

embarked = data[data['Survived']
== 0]['Embarked_S'].sum()
    print(f"1
{non_survivors_by_embarked}")e
lse: print("\nColumn
'Embarked_S' not found. Ensure
one-hot encoding includes
it.")print("0  122") print("Name:
Embarked_S, dtype: int64")# 5.
Calculate the percentage of
children (Age < 18) who
survivedchildren =
data[data['Age'] < 18]if not
children.empty:
    children_survival_rate =
(children['Survived'].sum() /
len(children))
    print(f"{children_survival_ra
te:}")else:    print("\nNo data
available for children.")# 6.
Calculate the percentage of
adults (Age >= 18) who
survivedadults = data[data['Age']
>= 18]if not adults.empty:
    adults_survival_rate =
(adults['Survived'].sum() /
len(adults))
    print(f"{adults_survival_rate
:}")else:  print("\nNo data
available for adults.")# 7. Get the
median age of
survivorsmedian_age_survivors
= data[data['Survived'] ==
1]['Age'].median()print(f"{median
_age_survivors}")# 8. Get the
median age of

```

```

non-
survivorsmedian_age_non_survi
vors = data[data['Survived'] ==
0]['Age'].median()print(f"{median
_age_non_survivors}")# 9. Get
the median fare of
survivorsmedian_fare_survivors
= data[data['Survived'] ==
1]['Fare'].median()print(f"{media
n_fare_survivors}")median_fare_
non_survivors =
data[data['Survived'] ==
0]['Fare'].median()print(f"{media
n_fare_non_survivors}")# 10. Get
the median fare of non-survivors

```

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked

Sample Data:

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['ParCh']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9 # 1. Calculate the survival rate by class
10 survival_rate_by_class = data.groupby('Pclass')['Survived'].mean()
11 print(survival_rate_by_class)
12 # 2. Calculate the survival rate by embarked location
13 if 'Embarked_S' in data.columns:
14     survival_rate_by_embarked = data.groupby('Embarked_S')
15     ['Survived'].mean()
16     print(survival_rate_by_embarked)
17 else:
18     print("\nColumn 'Embarked_S' not found. Ensure one-hot encoding
19     include it.")
20 # 3. Calculate the survival rate by family size
21 survival_rate_by_family_size = data.groupby('FamilySize')
22 ['Survived'].mean()
23 print(survival_rate_by_family_size)
24 # 4. Calculate the survival rate by being alone

```

5.2.1

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset from the CSV file
df = pd.read_csv('titanic.csv')

# Set up the figure for 5 subplots
fig, axes = plt.subplots(3, 2, figsize=(12, 12))

# write the code..
count_P = df["Pclass"].value_counts().sort_index()
count_G = df["Gender"].value_counts()
count_S = df["Survived"].value_counts().sort_index()

axes[0,0].bar(count_P.index, count_P.values, color=['skyblue'])
axes[0,0].set_title("Passenger Class Distribution")
axes[0,0].set_xlabel("Pclass")
axes[0,0].set_ylabel("Count")

axes[0,1].pie(count_G.values, labels=['male','female'], colors=['lightblue','lightcoral'], autopct='%1.1f%%')
axes[0,1].set_title("Gender Distribution")

axes[1,0].hist(df['Age'], color=['lightgreen'], edgecolor='black', bins=8)
axes[1,0].set_title("Age Distribution")
axes[1,0].set_xlabel("Age")
axes[1,0].set_ylabel("Frequency")

axes[1,1].bar(count_S.index, count_S.values, color=['lightblue','lightcoral'])
axes[1,1].set_title("Survival Count")
axes[1,1].set_xlabel("Survived(0=NO,1=Yes)")
axes[1,1].set_ylabel("Count")
```

```

Count')axes[1,1].set_xlabel('Survived(0=NO,1=Yes)')axes[1,1].set_ylabel('Count')axes[2,0].scatter(df['Age'],df['Fare'],color='orange')axes[2,0].set_title('Fare vs Age')axes[2,0].set_xlabel('Age')axes[2,0].set_ylabel('Fare')plt.tight_layout()plt.show

```

5.2.1. Titanic Dataset

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

Dataset Information:

The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- `Pclass`: Passenger class (1 = First, 2 = Second, 3 = Third).
- `Gender`: Gender of the passenger (male/female).
- `Age`: Age of the passenger.
- `Survived`: Survival status (0 = Did not survive, 1 = Survived).
- `Fare`: Ticket fare paid by the passenger.

Visualization:

To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

Visualization Details:

Write the code to create a series of visualizations as follows:

Bar Plot (`Pclass` Distribution):

- Create a bar plot to show the distribution of passengers across the different passenger classes (`Pclass`).
- Use the color `skyblue` for the bars.

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset from the CSV file
5 df = pd.read_csv('titanic.csv')
6
7 # Set up the figure for 5 subplots
8 fig, axes = plt.subplots(3, 2, figsize=(12,
9
10 # write the code..
11 count = df['Pclass'].value_counts().sort in

```

Average time

1.099 s
1099.00 ms

Maximum time

1.099 s
1099.00 ms

✓ 1 out of 1 shown test

✓ Test case 1 1099 ms

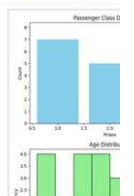
Debug

Test cases

Expected output



Actual output



Terminal

Test cases

5.2.2

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Histogram
plt.hist(data['Age'], bins=30, edgecolor='k')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Distribution')
plt.show()
```

5.2.1. Titanic Dataset

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

Dataset Information:

The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- `Pclass`: Passenger class (1 = First, 2 = Second, 3 = Third).
- `Gender`: Gender of the passenger (male/female).
- `Age`: Age of the passenger.
- `Survived`: Survival status (0 = Did not survive, 1 = Survived).
- `Fare`: Ticket fare paid by the passenger.

Visualization:

To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

Visualization Details:

Write the code to create a series of visualizations as follows:

Bar Plot (`Pclass` Distribution):

- Create a bar plot to show the distribution of passengers across the different passenger classes (`Pclass`).
- Use the color `skyblue` for the bars.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset from the CSV file
5 df = pd.read_csv('titanic.csv')
6
7 # Set up the figure for 5 subplots
8 fig, axes = plt.subplots(3, 2, figsize=(12,
9
10 # write the code..
11 count = df["Pclass"].value_counts().sort in
```

Average time

1.099 s

1099.00 ms

Maximum time

1.099 s

1099.00 ms

✓ 1 out of 1 shown test

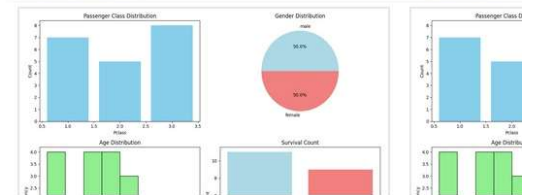
✓ Test case 1

1099 ms

Debug

Test cases

Expected output



Terminal

Test cases

5.2.3

```
import pandas as pd
import matplotlib.pyplot as plt
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Write your code here for Bar Plot for Survival Rates
survival_counts = data['Survived'].value_counts()
survival_counts.plot(kind='bar')
plt.title('Survival Count')
plt.xlabel('Survived')
plt.ylabel('Count')
plt.show()
```

CODETANTRA

HomeLearn Anywhere

202501080115@mitaoe.ac.inSupportLogout

5.2.2. Histogram of passenger information of Titanic

09:15

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:
1. Use 30 bins for the histogram.
2. Set the edge color of the bars to black (k).
3. Label the x-axis as 'Age' and the y-axis as 'Frequency'.
4. Add the title "Age Distribution" to the histogram.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

Histogram...

Submit

1import pandas as pd
2import matplotlib.pyplot as plt
3
4# Load the Titanic dataset
5data = pd.read_csv('Titanic-Dataset.csv')
6
7# Data Cleaning
8data['Age'].fillna(data['Age'].median(), inplace=True)
9data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10data.drop('Cabin', axis=1, inplace=True)
11

Average time0.245 s245.00 msMaximum time0.245 s245.00 ms1 out of 1 shown test case(s) passed

Test case 1245 msDebug

Expected outputActual output

Age DistributionAge Distribution

TerminalTest cases

< PrevResetSubmitNext >

5.2.4

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Gender

# Write your code here for Bar Plot for Survival by Gender

survival_counts = data.groupby('Sex')['Survived'].value_counts().unstack()

survival_counts.plot(kind='bar', stacked=True)

plt.title('Survival by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived'])
plt.show()
```

5.2.3. Bar plot of survival rate of passengers

25.43

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:

1. Use the **'Survived'** column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to **'bar'**.
3. Add the title **"Survival Count"** to the chart.
4. Label the x-axis as **'Survived'** and the y-axis as **'Count'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master Gosta Leonard",male,2,3,1,340909,21.075,,S
```

BarPlotOf...

Submit

```
1 # import pandas as pd
2 # import matplotlib.pyplot as plt
3
4 ## Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 ## Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
```

Average time
0.297 s
297.00 ms

Maximum time
0.297 s
297.00 ms

1 out of 1 shown test case(s) passed

Test case 1 297 ms Debug

Expected output

Actual output

Terminal

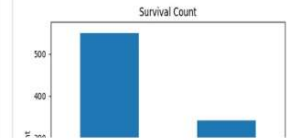
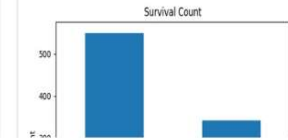
Test cases

< Prev

Reset

Submit

Next >



5.2.5

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Bar Plot for Survival by Pclass
survival_counts = data.groupby('Pclass')['Survived'].value_counts().unstack().fillna(0)
survival_counts.plot(kind='bar', stacked=True)

plt.title('Survival by Pclass')
plt.xlabel('Pclass')
plt.ylabel('Count')
plt.legend(['Not Survived', 'Survived'])
plt.show()
```

5.2.4. Bar Plot for Survival by Gender

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender, in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the 'Sex' column, then use the `value_counts()` function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title "Survival by Gender" to the chart.
4. Label the x-axis as 'Gender' and the y-axis as 'Count'.
5. The legend should indicate 'Not Survived' and 'Survived'.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
```

Explorer

BarPlotOf...

```
1 import pandas as pd
2 # import matplotlib.pyplot as plt
3
4 ## Load the Titanic dataset
5 # data = pd.read_csv('Titanic-Dataset.csv')
6
7 ## Data Cleaning
8 # data['Age'].fillna(data['Age'].median(), inplace=True)
9 # data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 # data.drop('Cabin', axis=1, inplace=True)
11
12 ## Convert categorical features to numeric
13 # data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 # data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15 # Write your code here for Bar Plot for Survival by Gender
16 import matplotlib.pyplot as plt
17 data = pd.read_csv('Titanic-Dataset.csv')
18 data['Age'].fillna(data['Age'].median(), inplace=True)
19 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
20 data.drop('Cabin', axis=1, inplace=True)
21 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
22 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
23
24 survival_counts = data.groupby('Sex')
25     ['Survived'].value_counts().unstack()
```

Terminal Test cases

< Prev

5.2.7

```
import pandas as pd
import matplotlib.pyplot as plt
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Write your code here for Box Plot for Age by Pclass
data.boxplot(column = 'Age', by='Pclass')
plt.title('Age by Pclass')
plt.suptitle("")
plt.xlabel('Pclass')
plt.ylabel('Age')
plt.show()
```

5.2.7. Box plot for Age Distribution

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,340920,21.075,C
```

BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Age by Pclass
17 data.boxplot(column = 'Age', by='Pclass')
18 plt.title('Age by Pclass')
19 plt.suptitle('')
20 plt.xlabel('Pclass')
21 plt.ylabel('Age')
22 plt.show()
23
24
25
```

Terminal Test cases

5.2.8

```
import pandas as pd
import matplotlib.pyplot as plt
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Write your code here for Box Plot for Age by Survived
plt.figure(figsize=(8,6))
data.boxplot(column='Age',by='Survived')
plt.title('Age by Survival')
plt.suptitle('')
plt.xlabel('Survived')
plt.ylabel('Age')
plt.show()
```

5.2.7. Box plot for Age Distribution

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,340920,21.075,C
```

BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Age by Pclass
17 data.boxplot(column = 'Age', by='Pclass')
18 plt.title('Age by Pclass')
19 plt.suptitle('')
20 plt.xlabel('Pclass')
21 plt.ylabel('Age')
22 plt.show()
23
24
25
```

Terminal Test cases

5.2.9

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)

# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# Write your code here for Box Plot for Fare by Pclass
plt.figure(figsize=(8,6))
data.boxplot(column='Fare', by='Pclass')
plt.title('Fare by Pclass')
plt.suptitle('')
plt.xlabel('Pclass')
plt.ylabel('Fare')
plt.show()
```

5.2.8. Box Plot for Age by Survived

00:29

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

1. Use the **Survived** column to group the data for the boxplot (0 = Did not survive, 1 = Survived).
2. Set the title of the plot to **"Age by Survival"**.
3. Remove the default subtitle with `plt.suptitle('')`.
4. Label the x-axis as **'Survived'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
```

BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numerical
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'])
15
16 # Write your code here for Box Plot for Age by Survival
17 # Write your code here for Box Plot for Age by Survival
18 plt.figure(figsize=(8,6))
19 data.boxplot(column='Age', by='Survived')
20 plt.title('Age by Survival')
21 plt.suptitle('')
22 plt.xlabel('Survived')
23 plt.ylabel('Age')
24 plt.show()
25
```

Terminal Test cases

31°C
Mostly sunny



Search



5.2.10

```
import pandas as pd
import matplotlib.pyplot as plt
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Write your code here for Box Plot for Fare by Pclass
plt.figure()
plt.scatter(data['Age'], data['Fare'])
plt.title('Age vs. Fare')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

5.2.9. Box Plot for Fare by Pclass

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Fare by Pclass"**.
3. Remove the default subtitle with **plt.suptitle("")**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, S
2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.2833, C85, C
3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2. 3101282, 7.925, S
4, 1, 1, "Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 113803, 53.1, C123, S
```

BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
```

Average time

0.267 s

267.00 ms

Maximum time

0.267 s

267.00 ms

1 out of 1 shown test case(s) passed

Test case 1

267 ms

Debug

Expected output

Actual output

5.2.11

```
import pandas as pd
import matplotlib.pyplot as plt
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# Data Cleaning
data['Age'].fillna(data['Age'].median(),
inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
data.drop('Cabin', axis=1, inplace=True)
# Convert categorical features to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
# Write your code here for Scatter Plot for Age vs. Fare by Survived
colors = data['Survived'].map({0: 'red', 1: 'blue'})
plt.scatter(data['Age'], data['Fare'], c=colors)
plt.title('Age vs. Fare by Survival')
plt.xlabel('Age')
plt.ylabel('Fare')
plt.show()
```

2.10. Scatter Plot for Age vs. Fare

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to **"Age vs. Fare"**.
3. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
```

AgeFareS...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
```

Average time
0.297 s
297.00 ms

Maximum time
0.297 s
297.00 ms

1 out of 1 shown test case(s) passed

Test case 1 297 ms

Debug

Expected output



Actual output



Terminal

Test cases

THANK YOU!